

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Naveen Kumar S	Reg. No.:	192421030
Section / Batch:	B Tech IT	Date:	25.08.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Focus Area	Questions
Comparative Analysis	Comparative analysis of Dynamic Programming and Greedy algorithms for optimization problems.	Q1

SECTION A – Comparative Analysis

Code of Conduct

I Naveen Kumar S - 192421030 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Naveen Kumar S

RUBRICS FOR CALCULATION

Comparative Analysis

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%				
Comparison & Differentiation	25%				
Critical Evaluation	25%				
Synthesis & Insight Generation	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
-------------------	--------------------	--------------------

Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

The screenshot shows the SIMATS Online Programming Lab interface. The header includes the SIMATS ENGINEERING logo, the title "SIMATS Online Programming Lab", and the course "DAA PROGRAMMING". The user's name "NAVEEN KUMAR S" and ID "192421030RC" are displayed in the top right.

The main area is divided into three sections:

- Your Code:** A Python code editor containing the following code:


```

1 def knapsack_dp(weights, values, capacity, n):
2     dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]
3
4     for i in range(1, n + 1):
5         for w in range(capacity + 1):
6             if weights[i - 1] <= w:
7                 dp[i][w] = max(dp[i][w], dp[i - 1][w] + values[i - 1])
8             else:
9                 dp[i][w] = dp[i - 1][w]
10
11     selected_items = []
12     w = capacity
13     for i in range(n, 0, -1):
14         if dp[i][w] != dp[i - 1][w]:
15             selected_items.append(i - 1)
16             w -= weights[i - 1]
17
18     return dp[n][capacity], selected_items
19
20
21
22 def container_loading_greedy(weights, values, capacity, n):
23     ratio = [(values[i] / weights[i], i) for i in range(n)]
24     ratio.sort(reverse=True)
25
26     selected_items = []
27     w = 0
28     for i in range(n):
29         if w + weights[ratio[i][1]] <= capacity:
30             selected_items.append(ratio[i][1])
31             w += weights[ratio[i][1]]
32
33     return w, selected_items

```
- Input:** A text area containing the following input:


```

4
2 3 4 5
3 4 5 6
5

```
- Output:** A window showing the results of the execution:


```

=== 0/1 Knapsack (DP) ===
Selected Items (indices): []
Maximum Value: 0

=== Container Loading (Greedy) ===

```

EXPLANATION :

The question asks you to **solve and compare two optimization methods: 0/1 Knapsack using Dynamic Programming (DP) and Container Loading using a Greedy approach.**

- **0/1 Knapsack (DP):** Each item can either be selected or not selected. The goal is to **maximize the total value without exceeding the given capacity**. Dynamic Programming checks different combinations systematically and guarantees the optimal solution.
- **Container Loading (Greedy):** Items are selected based on a rule such as **highest value-to-weight ratio** or best utilization first. It is usually faster, but it may not produce the optimal solution because selecting the best item at each step can prevent a better combination later.
- **Comparison:** You should compare the **items selected, total weight, total value, and capacity utilization** of both methods.

- **Why Greedy can fail:** Knapsack is a **discrete optimization problem**, where items cannot be divided. A locally best choice may leave unused capacity and result in a lower total value than another combination.
- **Complexity:** DP for 0/1 Knapsack generally has $O(nW)$ time complexity, where n is the number of items and W is the capacity. A typical greedy approach requires $O(n \log n)$ if items are sorted by value/weight ratio.
- **Real-world applications:** These methods are useful in **cargo transportation, warehouse management, delivery planning, resource allocation, and selecting products for limited storage space.**

In short: DP focuses on finding the **best possible combination**, while Greedy focuses on making the **best immediate choice**. Greedy is faster but does not always guarantee the optimal solution for 0/1 Knapsack.